

Secure Secrets, the CLI Way

Student Lab: Azure Key Vault, Blob Storage & a .NET API

All commands use Windows Command Prompt (cmd.exe)

Lab Overview

In this lab you will build a complete, security-first cloud application entirely from the command line. By the end you will have an Azure Key Vault storing a Blob Storage connection string, and an ASP.NET Core Web API that retrieves that secret at startup using a Managed Identity — no passwords, no hard-coded credentials, and no Portal clicking.

What You Will Build

Lab	Goal
Lab 1: CLI Setup	Sign in, set variables, create the resource group
Lab 2: Key Vault	Create an RBAC-enabled vault and test secret read/write
Lab 3: Blob Storage	Create a storage account and store the connection string in Key Vault
Lab 4: .NET API Scaffold	Scaffold an ASP.NET Core Web API and add the Azure packages
Lab 5: Key Vault Integration	Wire Key Vault into the app configuration using DefaultAzureCredential
Lab 6: Blob Endpoints	Build upload and download endpoints that use the Key-Vault-sourced connection string
Lab 7: Deploy with Managed Identity	Deploy the API to App Service and grant it passwordless access to Key Vault
Lab 8: Cleanup	Delete all resources with a single command

Before You Start

- Open a Windows Command Prompt (cmd.exe). Look for plain C:\...> at the prompt — if you see PS C:\...> you are in PowerShell and the variable syntax will be different.
- Confirm the Azure CLI is installed: `az --version`
- Confirm the .NET SDK is installed: `dotnet --version` (8.0 or later required)
- You need an active Azure subscription.

Tip

All variables you set with the `set` command only last for the current session. Keep this `cmd.exe` window open for the whole lab. If you close it and reopen, you will need to re-run the `set` commands from Lab 1.

Important

Key Vault names and Storage account names must be globally unique across all of Azure. Use the `%RANDOM%` trick in Lab 1 to generate a unique suffix — do not change it once you have set it.

Lab 1: CLI Setup

Step 1 — Sign In

1. Run the following command. A browser window will open for you to log in.

```
az login
```

```
mohamedabubakkars ~ % az login
A web browser has been opened at https://login.microsoftonline.com/organizations/oauth2/v2.0/authorize. Please continue the login in the web browser. If no web browser is available or if the web browser fails to open, use device code flow with `az login --use-device-code`.

Retrieving tenants and subscriptions for the selection...
The following tenants don't contain accessible subscriptions. Use `az login --allow-no-subscriptions` to have tenant level access.
7f7268a4-7179-4a42-9837-40255b1c21e5 'Presidio Network Solutions'

[Tenant and subscription selection]

No      Subscription name      Subscription ID      Tenant
-----
[1] *   Training-2026         e25db867-f63d-489a-8bce-7de21e5f94a1  Presidio Networked Solutions

The default is marked with an *; the default tenant is 'Presidio Networked Solutions' and subscription is 'Training-2026' (e25db867-f63d-489a-8bce-7de21e5f94a1).

Select a subscription and tenant (Type a number or Enter for no changes): 1

Tenant: Presidio Networked Solutions
Subscription: Training-2026 (e25db867-f63d-489a-8bce-7de21e5f94a1)

[Announcements]
With the new Azure CLI login experience, you can select the subscription you want to use more easily. Learn more about it and its configuration at https://go.microsoft.com/fwlink/?linkid=2271236

If you encounter any problem, please open an issue at https://aka.ms/azclibug

[Warning] The login output has been updated. Please be aware that it no longer displays the full list of available subscriptions by default.

mohamedabubakkars ~ %
```

Step 2 — Set Your Variables

2. Type each command exactly as shown. Do not add spaces around the = sign.

```
# Set reusable variables
export RG="cli-lab-rg"
export LOCATION="eastus"

# Random suffix
SUFFIX=$RANDOM

export KV_NAME="cli-lab-kv-$SUFFIX"
export SA_NAME="clilabsa$SUFFIX"
export CONTAINER_NAME="uploads"
```

3. Confirm each variable was set correctly:

```
echo $RG
echo $KV_NAME
echo $SA_NAME
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % echo $RG
cli-lab-rg
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % echo $KV_NAME
cli-lab-kv-22008
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % echo $SA_NAME
clilabsa22008
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % █
```

Write your values here for reference during the rest of the lab:

KV_NAME value	cli-lab-kv-22008
----------------------	------------------

SA_NAME value	clilabsa22008S
----------------------	----------------

SUFFIX value	22008
---------------------	-------

Step 3 — Create the Resource Group

```
az group create --name %RG% --location %LOCATION%
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az group create \
  --name "$RG" \
  --location "$LOCATION"
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abu-cli-lab-rg",
  "location": "eastus",
  "managedBy": null,
  "name": "abu-cli-lab-rg",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % █
```


Lab 2: Create and Secure a Key Vault

Step 1 — Create the Vault

4. Run the following command. The ^ symbol continues the command on the next line — type it exactly as shown.

```
az keyvault create ^
--name %KV_NAME% ^
--resource-group %RG% ^
--location %LOCATION% ^
--enable-rbac-authorization false
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az group create \
--name "$RG" \
--location "$LOCATION"
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abu-cli-lab-rg",
  "location": "eastus",
  "managedBy": null,
  "name": "abu-cli-lab-rg",
  "properties": {
    "provisioningState": "Succeeded"
  },
  "tags": null,
  "type": "Microsoft.Resources/resourceGroups"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az keyvault create \
--name "$KV_NAME" \
--resource-group "$RG" \
--location "$LOCATION" \
--enable-rbac-authorization false
{
  "id": "/subscriptions/e25db867-f63d-489a-8bce-7de21e5f94a1/resourceGroups/abu-cli-lab-rg/providers/Microsoft.KeyVault/vaults/abu-cli-lab-kv-6964",
  "location": "eastus",
  "name": "abu-cli-lab-kv-6964",
  "properties": {
    "accessPolicies": [
      {
        "applicationId": null,
        "objectId": "47f0ea90-47ee-46f3-afaf-2ccec0aef784",
        "permissions": {
          "certificates": [
            "all"
          ]
        }
      }
    ]
  }
}
```

Step 2 — Grant Yourself Access

5. Capture your user principal name and subscription ID, then create the role assignment:

```
for /f "delims=" %a in ('az ad signed-in-user show --query userPrincipalName -o tsv') do set MY_UPN=%a
for /f "delims=" %a in ('az account show --query id -o tsv') do set SUBSCRIPTION_ID=%a

az role assignment create ^
--role "Key Vault Secrets Officer" ^
--assignee %MY_UPN% ^
--scope
"/subscriptions/%SUBSCRIPTION_ID%/resourceGroups/%RG%/providers/Microsoft.KeyVault/vaults/%KV_NAME%"
```

Tip

The for /f loop captures command output into a variable. Use single % signs when typing at the prompt (not inside a .bat file).

Important

Role assignments can take 1–2 minutes to propagate. If the next step returns a 403 error, wait a minute and try again.

**Screenshot**

Role assignment created — JSON output showing roleDefinitionName: Key Vault Secrets Officer

Step 3 — Smoke-Test the Vault

- Set a test secret and read it back:

```
az keyvault secret set --vault-name %KV_NAME% --name "HelloWorld" --value "it-works"
az keyvault secret show --vault-name %KV_NAME% --name "HelloWorld" --query value -o tsv
```

The second command should print: it-works

- Delete the test secret:

```
az keyvault secret delete --vault-name %KV_NAME% --name "HelloWorld"
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az keyvault secret set \
--vault-name "$KV_NAME" \
--name "HelloWorld" \
--value "it-works"
{
  "attributes": {
    "created": "2026-07-01T09:39:43+00:00",
    "enabled": true,
    "expires": null,
    "notBefore": null,
    "recoverableDays": 90,
    "recoveryLevel": "Recoverable+Purgeable",
    "updated": "2026-07-01T09:39:43+00:00"
  },
  "contentType": null,
  "id": "https://abu-cli-lab-kv-6964.vault.azure.net/secrets/HelloWorld/f003ea3ee3724b64ae569484902e3829",
  "kid": null,
  "managed": null,
  "name": "HelloWorld",
  "tags": {
    "file-encoding": "utf-8"
  },
  "value": "it-works"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az keyvault secret show \
--vault-name "$KV_NAME" \
--name "HelloWorld" \
--query value \
-o tsv
it-works
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ %
```

Lab 3: Blob Storage and the Connection String Secret

Step 1 — Create the Storage Account and Container

```
az storage account create ^
--name %SA_NAME% ^
--resource-group %RG% ^
--location %LOCATION% ^
--sku Standard_LRS ^
--kind StorageV2

az storage container create ^
--name %CONTAINER_NAME% ^
--account-name %SA_NAME% ^
--auth-mode login
```

```
web": "https://abuc11lab5a6984.213.w
},
"primaryLocation": "eastus",
"privateEndpointConnections": [],
"provisioningState": "Succeeded",
"publicNetworkAccess": null,
"resourceGroup": "abu-cli-lab-rg",
"routingPreference": null,
"sasPolicy": null,
"secondaryEndpoints": null,
"secondaryLocation": null,
"sku": {
  "name": "Standard_LRS",
  "tier": "Standard"
},
"statusOfPrimary": "available",
```

Step 2 — Capture the Connection String

```
for /f "delims=" %a in ('az storage account show-connection-string --name %SA_NAME%
--resource-group %RG% --query connectionString -o tsv') do set CONN_STRING=%a
```

Tip

Verify the variable was captured: `echo %CONN_STRING%` — it should start with `DefaultEndpointsProtocol=https`

Step 3 — Store the Connection String in Key Vault

```
az keyvault secret set ^
--vault-name %KV_NAME% ^
--name "BlobStorageConnectionString" ^
--value "%CONN_STRING%"
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ % az keyvault secret set \
--vault-name "$KV_NAME" \
--name "BlobStorageConnectionString" \
--value "$CONN_STRING" \
{
  "attributes": {
    "created": "2026-07-01T09:53:40+00:00",
    "enabled": true,
    "expires": null,
    "notBefore": null,
    "recoverableDays": 90,
    "recoveryLevel": "Recoverable+Purgeable",
    "updated": "2026-07-01T09:53:40+00:00"
  },
  "contentType": null,
  "id": "https://abu-cli-lab-kv-6964.vault.azure.net/secrets/BlobStorageConnectionString/e3d857d887a342f582f812e3a5e0bbf2",
  "kid": null,
  "managed": null,
  "name": "BlobStorageConnectionString",
  "tags": {
    "file-encoding": "utf-8"
  },
  "value": "DefaultEndpointsProtocol=https;EndpointSuffix=core.windows.net;AccountName=abuclilabsa6964;AccountKey=wKfoV0ROwoGW5x9i5ayYW52rPB4QwSXL3pvsjGJtkZLwB+Te/beqUXG2WBkUdl9CsiPa8NPNcDyn+AST/qC2pA==;BlobEndpoint=https://abuclilabsa6964.blob.core.windows.net;/FileEndpoint=https://abuclilabsa6964.file.core.windows.net;/QueueEndpoint=https://abuclilabsa6964.queue.core.windows.net;/TableEndpoint=https://abuclilabsa6964.table.core.windows.net/"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars ~ %
```

Important

This is the last time the connection string will be handled directly. From here on, only Key Vault knows it.

Lab 4: Scaffold the .NET Web API

Step 1 — Create the Project

Tip

Choose a folder on your machine where you want to create the project, then cd into it before running these commands.

```
dotnet new webapi -n BlobVaultApi --use-controllers
cd BlobVaultApi
```

Step 2 — Add the Azure NuGet Packages

```
dotnet add package Azure.Identity
dotnet add package Azure.Extensions.AspNetCore.Configuration.Secrets
dotnet add package Azure.Storage.Blobs
```

```
<ItemGroup>
  <PackageReference Include="Azure.Extensions.AspNetCore.Configuration.Secrets" Version="1.5.1" ✓ />
  <PackageReference Include="Azure.Identity" Version="1.21.0" />
  <PackageReference Include="Azure.Storage.Blobs" Version="12.29.1" />
  <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="10.0.7" />
</ItemGroup>
```

Package	Purpose
Azure.Identity	DefaultAzureCredential — uses your az login locally, Managed Identity in Azure
Azure.Extensions.AspNetCore.Configuration.Secrets	Maps Key Vault secrets directly into ASP.NET Core configuration
Azure.Storage.Blobs	Blob Storage client for upload and download

Lab 5: Wire Up Key Vault in Program.cs

Step 1 — Edit Program.cs

8. Open BlobVaultApi/Program.cs in a text editor or IDE.
9. Replace the entire file content with the following:

```
using Azure.Identity;

var builder = WebApplication.CreateBuilder(args);

var keyVaultUri = new
Uri($"https://{builder.Configuration["KeyVaultName"]}.vault.azure.net/");
builder.Configuration.AddAzureKeyVault(keyVaultUri, new DefaultAzureCredential());

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();
app.UseSwagger();
app.UseSwaggerUI();
app.UseAuthorization();
app.MapControllers();
app.Run();
```

Step 2 — Set the Vault Name in App Settings

10. Open BlobVaultApi/appsettings.Development.json and replace the KeyVaultName value with YOUR vault name (the value you wrote down from Lab 1):

```
{
  "KeyVaultName": "YOUR-KV-NAME-HERE",
  "Logging": { "LogLevel": { "Default": "Information" } }
}
```

Important

Replace YOUR-KV-NAME-HERE with the actual value from your KV_NAME fill-in box on page 2. Using the wrong name will cause a connection error when you run the app.

Step 3 — Verify the App Starts

```
dotnet run
```

Watch the startup log. You should see no errors. The app will print a localhost URL — leave it running.

```

○ mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi % dotnet run
Using launch settings from /Users/mohamedabubakkars/GenSpark/Day-49/BlobVaultApi/Properties/launchSettings.json...
Building...
info: Microsoft.Hosting.Lifetime[14]
    Now listening on: http://localhost:5118
info: Microsoft.Hosting.Lifetime[0]
    Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
    Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
    Content root path: /Users/mohamedabubakkars/GenSpark/Day-49/BlobVaultApi

```

Lab 6: Blob Upload and Download Endpoints

Step 1 — Register the Blob Service Client

11. Add the following to Program.cs, after the AddAzureKeyVault line and before builder.Build():

```

using Azure.Storage.Blobs;

builder.Services.AddSingleton(x =>
{
    var connStr = builder.Configuration["BlobStorageConnectionString"];
    return new BlobServiceClient(connStr);
});

```

Step 2 — Create the Files Controller

12. Create a new file: BlobVaultApi/Controllers/FilesController.cs
13. Paste in the following code:

```

using Microsoft.AspNetCore.Mvc;
using Azure.Storage.Blobs;

namespace BlobVaultApi.Controllers;

[ApiController]
[Route("api/[controller]")]
public class FilesController : ControllerBase
{
    private readonly BlobContainerClient _container;
}

```



```

private const string ContainerName = "uploads";

public FilesController(BlobServiceClient blobServiceClient)
{
    _container = blobServiceClient.GetBlobContainerClient(ContainerName);
    _container.CreateIfNotExists();
}

[HttpPost("upload")]
public async Task<IActionResult> Upload(IFormFile file)
{
    if (file is null || file.Length == 0)
        return BadRequest("No file provided.");

    var blobClient = _container.GetBlobClient(file.FileName);
    using var stream = file.OpenReadStream();
    await blobClient.UploadAsync(stream, overwrite: true);

    return Ok(new { fileName = file.FileName, url = blobClient.Uri.ToString() });
}

[HttpGet("download/{fileName}")]
public async Task<IActionResult> Download(string fileName)
{
    var blobClient = _container.GetBlobClient(fileName);
    if (!await blobClient.ExistsAsync())
        return NotFound();

    var download = await blobClient.DownloadContentAsync();
    return File(download.Value.Content.ToArray(), "application/octet-stream",
fileName);
}
}

```

Step 3 — Run and Test

14. Start the app:

```
dotnet run
```

15. Open a SECOND cmd.exe window (leave dotnet run running in the first). Test the upload endpoint — replace sample.jpg with any image or file you have:

```
curl -F "file=@sample.jpg" https://localhost:5001/api/files/upload -k
```

16. Test the download endpoint:

```
curl https://localhost:5001/api/files/download/sample.jpg -k -o downloaded.jpg
```

```

● mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi % curl -F "file=@unnamed.png" http://localhost:5118/api/files/upload -k
{"fileName":"unnamed.png","url":"https://abucilabsa6964.blob.core.windows.net/uploads/unnamed.png"}

```

```

● mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi % curl http://localhost:5118/api/files/download/unnamed.png -o downloaded.p
ng
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left   Speed
 100 293k    100 293k    0     0    98k    0 0:00:02  0:00:02 --:--:--  98k
○ mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi %

```

Lab 7: Deploy with a Managed Identity

Step 1 — Create the App Service Plan and Web App

```
set APP_NAME=cli-lab-api-%SUFFIX%
```

```
az appservice plan create ^
--name cli-lab-plan ^
--resource-group %RG% ^
--sku B1 ^
--is-linux
```

```
az webapp create ^
--name %APP_NAME% ^
--resource-group %RG% ^
--plan cli-lab-plan ^
--runtime "DOTNETCORE:8.0"
```

```

mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi % az webapp create \
--name "$APP_NAME" \
--resource-group "$RG" \
--plan cli-lab-plan \
--runtime "DOTNETCORE:8.0"
Webapp 'abu-cli-lab-api-6964' created. Deploy your code with: az webapp deploy
{
  "autoGeneratedDomainNameLabelScope": null,
  "availabilityState": "Normal",
  "clientAffinityEnabled": true,
  "clientAffinityPartitioningEnabled": null,
  "clientAffinityProxyEnabled": false,
  "clientCertEnabled": false,
  "clientCertExclusionPaths": null,
  "clientCertMode": "Required",
  "cloningInfo": null,
  "containerSize": 0,
  "customDomainVerificationId": "46807E15E4B3288A1BCC91EA4477FF9CF6E3CE676FACDEA0663A4E096E2CCC01",
  "dailyMemoryTimeQuota": 0,
  "daprConfig": null,
  "defaultHostName": "abu-cli-lab-api-6964.azurewebsites.net",
  "dnsConfiguration": {
    "dnsAltServer": null,
    "dnsLegacySortOrder": null,
    "dnsMaxCacheTimeout": null,
    "dnsRetryAttemptCount": null,
    "dnsRetryAttemptTimeout": null,
    "dnsServers": null
  },
  "enabled": true,

```

APP_NAME value

abu-cli-lab-api-6964.azurewebsites.net

Step 2 — Configure the App Setting

```
az webapp config appsettings set ^
  --name %APP_NAME% ^
  --resource-group %RG% ^
  --settings KeyVaultName=%KV_NAME%
```

Step 3 — Assign a Managed Identity

```
az webapp identity assign --name %APP_NAME% --resource-group %RG%
```

17. Capture the identity's principal ID:

```
for /f "delims=" %a in ('az webapp identity show --name %APP_NAME% --resource-group %RG%
--query principalId -o tsv') do set PRINCIPAL_ID=%a
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi % az webapp identity assign \
  --name "$APP_NAME" \
  [ --resource-group "$RG"
  {
    "principalId": "461cc5f9-fd37-494e-8c6b-1592db1d79f6",
    "tenantId": "2a731c61-a2b2-4661-8409-5b861cf40d0c",
    "type": "SystemAssigned",
    "userAssignedIdentities": null
  }
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi % █
```

Step 4 — Grant the Identity Key Vault Access

```
az role assignment create ^
  --role "Key Vault Secrets User" ^
  --assignee %PRINCIPAL_ID% ^
  --scope
  "/subscriptions/%SUBSCRIPTION_ID%/resourceGroups/%RG%/providers/Microsoft.KeyVault/vaults/%KV_NAME%"
```

Step 5 — Publish and Deploy

cmd.exe has no built-in zip command, so you will call PowerShell just for the compression step:

```
dotnet publish -c Release -o .\publish
cd publish
powershell -Command "Compress-Archive -Path * -DestinationPath ..\publish.zip -Force"
cd ..
```

```
az webapp deploy ^
--name %APP_NAME% ^
--resource-group %RG% ^
--src-path publish.zip ^
--type zip
```

```
Deployment has completed successfully.
You can visit your app at: http://abu-cli-lab-api-6964.azurewebsites.net
{
  "active": true,
  "author": "N/A",
  "author_email": "N/A",
  "build_summary": {
    "errors": [],
    "warnings": []
  },
  "complete": true,
  "deployer": "OneDeploy",
  "end_time": "2026-07-01T11:26:40.5862246Z",
  "id": "4e3ea266-3c9c-4c5d-8925-8d1fe91d50b4",
  "is_readonly": true,
  "is_temp": false,
  "last_success_end_time": "2026-07-01T11:26:40.5862246Z",
  "log_url": "https://abu-cli-lab-api-6964.scm.azurewebsites.net/api/deployments/4e3ea266-3c9c-4c5d-8925-8d1fe91d50b4/log",
  "message": "OneDeploy",
  "progress": "",
  "received_time": "2026-07-01T11:26:34.793342Z",
  "site_name": "abu-cli-lab-api-6964",
  "start_time": "2026-07-01T11:26:36.2159053Z",
  "status": 4,
  "status_text": "",
  "url": "https://abu-cli-lab-api-6964.scm.azurewebsites.net/api/deployments/4e3ea266-3c9c-4c5d-8925-8d1fe91d50b4"
}
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi %
```

Step 6 — Test in the Cloud

```
curl -F "file=@unnamed.png"
https://abu-cli-lab-api-6964.azurewebsites.net/api/files/upload
```

```
|https://abu-cli-lab-api-6964.azurewebsites.net/api/files/upload
{"fileName":"unnamed.png","url":"https://abuclilabsa6964.blob.core.windows.net/uploads/unnamed.png"}%
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi %
```

Lab 8: Cleanup

Important

This step permanently deletes all resources created today. Only run it when you are completely finished with the lab.

```
az group delete --name %RG% --yes --no-wait
```

```
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi % az group delete \
--name "abu-cli-lab-rg" \
--yes \
[ --no-wait
mohamedabubakkars@FVFG908RQ05F-mohamedabubakkars BlobVaultApi %
```

Reflection Questions

Answer the following questions to check your understanding of the lab.

1. Why is the Blob Storage connection string stored in Key Vault instead of in appsettings.json?

The Blob Storage connection string is stored in **Azure Key Vault** instead of `appsettings.json` to improve security. Key Vault securely stores sensitive information such as connection strings and secrets, preventing them from being exposed in source code or GitHub repositories. Applications can securely retrieve the secret at runtime using **Managed Identity** and `DefaultAzureCredential`, eliminating the need to hard-code credentials.

2. What is `DefaultAzureCredential`, and why does the same line of code work both locally and in Azure?

`DefaultAzureCredential` is an authentication class that automatically selects the best available authentication method based on where the application is running. Locally, it uses the Azure credentials from `az login` or Visual Studio. In Azure, it automatically uses the App Service's **Managed Identity**. This allows the same code to work in both local and Azure environments without changing credentials or hard-coding secrets.

3. Why does the App Service identity get the Key Vault Secrets User role instead of Secrets Officer?

The App Service is assigned the **Key Vault Secrets User** role because it only needs **read access** to retrieve secrets from Azure Key Vault. The **Secrets Officer** role provides additional permissions to create, update, and delete secrets, which violates the principle of least privilege. Granting only the required read permission improves security and reduces the risk of unauthorized changes.

4. What would happen if you accidentally committed appsettings.json with the raw connection string to a public GitHub repository? How does this lab's approach prevent that?

If the `appsettings.json` file containing the raw connection string were committed to a public GitHub repository, anyone could access the storage account and potentially read, modify, or delete data. In this lab, the connection string is stored securely in **Azure Key Vault**, and the application retrieves it at runtime using **Managed Identity** and `DefaultAzureCredential`. This keeps secrets out of the source code and prevents them from being exposed in version control.

Common Errors

Error	Fix
InvalidResourceGroup : name has invalid character '\$'	You are using PowerShell/bash variable syntax. Switch to cmd.exe and use %VAR% syntax
No such host known (vault.azure.net)	The vault name in appsettings.Development.json is still the placeholder. Replace it with your actual KV_NAME value
403 Forbidden (Key Vault)	Role assignment hasn't propagated yet — wait 60 seconds and retry
DefaultAzureCredential fails	Run az login again in the same cmd.exe window
curl upload returns 401/403	The App Service Managed Identity role assignment may not have propagated — wait 2 minutes